.NET Performance Optimization

Question 1: How to Improve Performance of .NET Applications?

1. Asynchronous Programming Use async/await for I/O-bound operations to prevent thread blocking. This allows your application to handle more concurrent requests without consuming additional threads from the thread pool. Asynchronous code improves scalability and responsiveness, especially in web applications handling multiple simultaneous requests.

2. Caching Strategies Implement caching at multiple levels to significantly reduce response times:

- **In-Memory Caching:** Store frequently accessed data in memory for instant retrieval
- **Distributed Caching:** Use Redis or SQL Server for scalable, shared cache across multiple servers
- **Output Caching:** Cache entire response output for static or semi-static content

Proper caching reduces database calls, computation overhead, and dramatically improves application responsiveness.

3. Connection Pooling Reuse database connections instead of creating new ones for each request.
ADO.NET handles this automatically, but you must properly dispose connections using using statements to return them to the pool. Connection pooling eliminates the expensive overhead of establishing new database connections.

4. Optimize LINQ Queries LINQ optimization is crucial for database-driven applications:

- Use compiled queries for repeated LINQ operations Prefer IQueryable over
- IEnumerable when working with databases to enable server-side filtering
- Avoid unnecessary .ToList() calls that materialize entire collections in memory
- Use projection to select only required columns

5. Memory Management Efficient memory usage prevents garbage collection pressure and improves performance:

- Minimize allocations in hot paths (frequently executed code)
- Use Span<T> and Memory<T> for efficient buffer management without heap allocations
- Properly dispose objects implementing IDisposable to release resources promptly
- Be cautious with large object heap allocations (objects > 85KB)
- Consider object pooling for frequently created/destroyed objects

6. Lazy Loading vs. Eager Loading Choose the appropriate data

loading strategy based on your scenario:

- **Eager Loading:** Use .Include() when you know you'll need related data to avoid N+1 query problems
- **Lazy Loading:** Use judiciously to defer loading until actually needed, but be aware of potential performance issues
- **Explicit Loading:** Manually control when related data is loaded for fine-grained control

7. Response Compression Enable response compression middleware in ASP.NET

Core to reduce payload
size over the network. This is especially effective for text-based responses like

JSON, HTML, and CSS, potentially reducing response sizes by 70-80%.

8. Profiling and Monitoring You cannot optimize what you cannot measure. Use

profiling tools to identify
actual bottlenecks:

- **Application Insights:** Real-time monitoring and analytics
- **MiniProfiler:** Lightweight profiler for database and application code
- **dotTrace/dotMemory:** JetBrains profiling tools for deep analysis
- **BenchmarkDotNet:** Accurate micro-benchmarking for performance testing

Key Takeaways for .NET Performance:

- Always measure before optimizing - profile to find real bottlenecks
- Async programming is essential for scalable I/O-bound operations
- Caching is one of the most effective performance improvements
- Memory efficiency directly impacts garbage collection and overall performance
- Choose the right data access patterns for your specific scenarios

Question 2: How to Improve Performance of Stored Procedures/SQL Queries?

1. Proper Indexing Indexing is the single most important factor in

query performance:

- Create indexes on columns used in WHERE, JOIN, and ORDER BY clauses
- Use covering indexes to include all columns needed by a query (eliminating key lookups)
- Consider composite indexes for queries filtering on multiple columns
- Regularly maintain indexes through reorganization (fragmentation 5-30%) and rebuilding (fragmentation > 30%)
- Monitor and remove unused indexes that only add overhead to DML operations

2. Avoid SELECT * Always specify only the columns you actually need. SELECT *

causes multiple
performance issues:

- Increases I/O by reading unnecessary data from disk
- Increases network traffic sending unnecessary data
- Increases memory consumption
- Prevents use of covering indexes in many cases
- Makes queries fragile to schema changes

3. Use WHERE Clauses Effectively Optimize

filtering to allow efficient index usage:

- Apply filters as early as possible in the query execution Use SARGable
- predicates (Search ARGument-able) that allow index usage Avoid
- functions on indexed columns in WHERE clauses (e.g., WHERE YEAR(DateColumn) = 2024 is not SARGable)
- Instead use: WHERE DateColumn >= '2024-01-01' AND DateColumn < '2025-01-01'

4. Minimize Table Scans Table scans read every row in a table and are extremely

expensive for large
tables. Check execution plans to identify table scans and add appropriate

indexes. Aim for index seeks instead of scans whenever possible.

5. Use SET NOCOUNT ON Add SET NOCOUNT ON at the beginning of stored

procedures to prevent SQL
Server from sending "rows affected" messages to the client after each statement.

This reduces network overhead and can improve performance, especially for procedures with many statements.

6. Avoid Cursors Cursors process data row-by-row and are dramatically slower than set-based
operations. SQL Server is optimized for set-based operations, not procedural row-by-row processing. Replace cursor logic with:

- JOIN operations Subqueries Common
- Table Expressions (CTEs) Window
- functions
- Temporary tables with set-based
- updates

7. Parameter Sniffing Awareness SQL Server creates execution plans based on the first parameter values used (parameter sniffing). This can cause performance issues when the plan isn't optimal for other parameter values:

- Use OPTION (RECOMPILE) for queries where parameter values vary widely
- Use local variables to disable parameter sniffing when needed Use
- OPTION (OPTIMIZE FOR) to hint at typical parameter values Consider plan
- guides for problematic queries you cannot modify

8. Update Statistics SQL Server's query optimizer relies on statistics to make informed decisions about execution plans. Keep statistics current by:

- Enabling auto-update statistics (enabled by default)
- Manually updating statistics after large data changes
- Using UPDATE STATISTICS WITH FULLSCAN for critical
- tables

Monitoring statistics staleness

9. Avoid Implicit Conversions Ensure data types match between variables and columns. Implicit conversions prevent index usage and add CPU overhead. For example, comparing VARCHAR columns to NVARCHAR parameters forces conversion and prevents index seeks.

10. Use EXISTS Instead of COUNT When checking for existence, use EXISTS rather than COUNT(*) > 0. EXISTS stops as soon as it finds a matching row, while COUNT must process all matching rows. This can save significant processing time on large datasets.

11. Limit Result Sets Use TOP, OFFSET-FETCH, or pagination to limit the number of rows returned,
especially in web applications. Returning thousands of rows wastes memory, network bandwidth, and processing time when users typically view only the first page of results.

12. Analyze Execution Plans Regularly review actual execution plans to identify expensive operations:

- **Table/Index Scans:** Often indicate missing indexes **Key**
- **Lookups:** May benefit from covering indexes **Sort**
- **Operations:** Consider indexes on ORDER BY columns
- **Hash Joins:** May indicate missing indexes on join columns
- **Implicit Conversions:** Warning signs in execution plans

Key Takeaways for SQL Performance:

- Proper indexing is the foundation of query performance
- Write SARGable queries that can leverage indexes effectively
- Think in sets, not rows - avoid cursors and row-by-row processing
- Always analyze execution plans to understand query behavior
- Regular maintenance (statistics, index fragmentation) is essential
- Select only what you need - avoid SELECT * and excessive columns